WEEK 4 · LESSON 20 OF 24 · HANDOUT

# Recurrent Networks and LSTMs for Sequences

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=YCzL96nL7j0 — open the lesson online to watch inside the course.

🎓 **Go deeper:** MIT 6.S191 (2024): Recurrent Neural Networks, Transformers and Attention — https://www.youtube.com/watch?v=dqoEU9Ac3ek

🎓 **Go deeper:** EuroPython: Understanding and Implementing Recurrent Neural Networks using Python — https://www.youtube.com/watch?v=bSdBG7hToOg

🎓 **Go deeper:** PyData LA: 1D Convolutional Neural Networks for Time Series Modelling — https://www.youtube.com/watch?v=nMkqWxMjWzg

**Your goal in this lesson:** Explain how recurrent networks carry memory through a sequence, why LSTMs were invented, and forecast a time series with an LSTM compared with honest baselines.

## Data that comes in order

Sales per day, rainfall per week, words in a sentence, heartbeats in an ECG — in a **sequence**, order carries meaning. A **recurrent neural network (RNN)** reads one step at a time and keeps a **hidden state**: a running summary of everything it has seen so far.

> *new hidden state = activation(W × current input + U × previous hidden state + b)*

The same weights are used at every step — like weight sharing in a CNN, but along time.

## The trouble with plain RNNs

Training an RNN means backpropagating through every time step — a very deep network. Gradients vanish (or explode) over long sequences, so a plain RNN forgets what happened more than a few steps ago.

The **LSTM** (Long Short-Term Memory, 1997) adds a **cell state** — a conveyor belt of memory — controlled by three **gates**:

| Gate | Decides |
|---|---|
| Forget gate | What to erase from memory |
| Input gate | What new information to store |
| Output gate | What to reveal as the hidden state |

Gates are small sigmoid layers whose outputs (0 to 1) act like taps. The **GRU** is a simpler cousin with two gates. Before transformers (2017), LSTMs powered speech recognition and machine translation.

## The data: daily electricity demand

We simulate two years of daily electricity demand (in megawatts) for a city: a weekly pattern (lower on weekends), slow growth, a yearly wave and noise.

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

rng = np.random.default_rng(4)
days = np.arange(730)
weekly = np.array([1.0, 1.02, 1.03, 1.03, 1.05, 0.92, 0.85])[days % 7]
demand = (150 + 0.04 * days) * weekly + 8 * np.sin(2 * np.pi * days / 365) + rng.normal(0, 3,
len(days))

plt.plot(demand)
plt.xlabel("day")
plt.ylabel("demand (MW)")
plt.title("Simulated daily electricity demand")
plt.show()
```

## Windows: turn a series into examples

Each example is the last **28 days**; the target is the **next** day. We train on the first 600 days and test on the rest — never shuffle time series across the split, or the model sees the future.

```
WINDOW = 28
mean, std = demand[:600].mean(), demand[:600].std()      # training statistics only
z = (demand - mean) / std

X = np.stack([z[i:i + WINDOW] for i in range(len(z) - WINDOW)])
y = z[WINDOW:]
split = 600 - WINDOW
X_train, y_train, X_test, y_test = X[:split], y[:split], X[split:], y[split:]
real = y_test * std + mean
print(X_train.shape, X_test.shape)
```

## Baselines first

```
yesterday = X_test[:, -1] * std + mean          # "tomorrow = today"
last_week = X_test[:, -7] * std + mean          # "tomorrow = same day last week"
print("MAE, same as yesterday:", round(np.abs(yesterday - real).mean(), 2), "MW")
print("MAE, same as last week:", round(np.abs(last_week - real).mean(), 2), "MW")
```

"Same as yesterday" is poor (about **11 MW** off) because of the weekend drop; "same day last week" is strong (about **3.5 MW**). Any model must beat **3.5**.

## An LSTM forecaster

```
class Forecaster(nn.Module):
    def __init__(self, hidden=32):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=hidden, batch_first=True)
        self.out = nn.Linear(hidden, 1)

    def forward(self, x):                    # x: (batch, 28 days)
        outputs, (h, c) = self.lstm(x.unsqueeze(-1))
        return self.out(outputs[:, -1]).squeeze(-1)   # use the state after the last day

t = lambda a: torch.tensor(a, dtype=torch.float32)
torch.manual_seed(0)
model = Forecaster()
opt = torch.optim.Adam(model.parameters(), lr=1e-2)
for epoch in range(60):
    model.train()
    perm = torch.randperm(len(X_train))
    for start in range(0, len(perm), 32):
        idx = perm[start:start + 32]
        opt.zero_grad()
        F.mse_loss(model(t(X_train[idx])), t(y_train[idx])).backward()
        opt.step()

model.eval()
with torch.no_grad():
    forecast = model(t(X_test)).numpy() * std + mean
print("MAE, LSTM:", round(np.abs(forecast - real).mean(), 2), "MW")

plt.plot(real, label="real")
plt.plot(forecast, label="LSTM forecast")
plt.legend()
plt.title("Test period: next-day forecasts")
plt.show()
```

The LSTM is off by about **2.9 MW**, beating the strong weekly baseline (about 3.5). It learned the weekly rhythm **and** the slower changes from the data, with no calendar given to it.

## Sequences beyond time series

- `nn.LSTM` also reads **text**: feed it the word embeddings from Lesson 19 instead of one number per day.
- **Bidirectional** LSTMs read forwards and backwards (useful for whole sentences).
- For many sequence tasks today, **transformers** (next lesson) are faster to train and more accurate, but LSTMs remain useful for small data and streaming signals on small devices.

## Key terms

- **Sequence** — data where order matters.
- **RNN** — a network that reads step by step and keeps a hidden state.
- **Hidden state** — the running memory of an RNN.
- **LSTM / GRU** — RNNs with gates that keep long-term memory.
- **Window** — a fixed number of past steps used as one example.
- **Seasonal baseline** — "same as last week": the forecast to beat.

**Remember this:** RNNs and LSTMs carry a memory along a sequence; for forecasting, build windows without peeking at the future and always beat a seasonal baseline.

Name: _____ Date: _____

## Activity

### Task 1

Replace `nn.LSTM` with `nn.GRU`. Is it better, worse or the same?

### Task 2

Use a 14-day window instead of 28. What happens to the error, and why?

### Task 3

Forecast 7 days ahead instead of 1 (change the target). How much harder is it?

### Task 4

Add the day of the week as a second input feature ( `input_size=2` ). Does the LSTM improve?

## Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does an RNN keep while reading a sequence?

   A. Only the last input
   B. Nothing
   C. The whole dataset
   D. A hidden state summarising what it has seen

2. Why were LSTMs invented?

   A. To replace embeddings
   B. To make networks smaller
   C. Plain RNNs forget long-range information because gradients vanish
   D. To process images

3. What does the forget gate of an LSTM decide?

   A. What to erase from memory
   B. Which word comes next
   C. The batch size
   D. The learning rate

4. Why must time-series data not be shuffled across the train/test split?

   A. The model would see the future, making the test dishonest
   B. It changes the units
   C. LSTMs cannot read shuffled data
   D. Shuffling is slow

5. Which baseline was strongest for daily demand with a weekly pattern?

   A. Same as yesterday
   B. Zero
   C. The overall mean
   D. Same day last week

6. What does batch_first=True mean in nn.LSTM?

   A. Shuffle batches
   B. Sort batches
   C. Train the first batch only
   D. The batch dimension comes first: (batch, time, features)

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.